



UNIVERSITY OF
PLYMOUTH

PUSL3190 Computing Project

Project Proposal

Local Browser Isolation With Download File
Activity Monitoring

Supervisor: Mr. Chamara Dissanayake

Name: Kotigampala Amaranath

Plymouth Index Number: 10953251

Degree Program: [BSc \(Hons\) Computer Security](#)

• **Table of Contents**

1. <u>Problem statement Introduction</u>	1
2. <u>Project description</u>	2
3. <u>Research Gap</u>	4
4. <u>Requirements Analysis</u>	5
5. <u>Finance</u>	6
6. <u>Time Frame / Timeline</u>	7
7. <u>Referencing / Bibliography</u>	8

● Problem statement

Web browsers have become the main entry point for both personal and professional activities. Every major activity General user and industrial people use Web browser.

However, browsers are also one of the biggest security risks. Cyber attackers often target browsers to steal data, track users, or inject malware into systems.

To solve this issue, many companies have adopted **Remote Browser Isolation (RBI)** technologies. RBI **allows users to open web pages on a remote server instead of their own computer. The content is then streamed safely to the user's device,** which helps protect against malware and phishing attacks. While this sounds good in theory, in practice it creates several problems.

This project aims to build an alternative solution called **Local Browser Isolation (LBI)**. Unlike RBI, this approach **isolates the browser within the user's own system, offering both security and privacy** without the high cost and performance issues of remote systems.

■ Industrial Problems

1. **Performance Overhead:** - VMs/containers add CPU/RAM usage, leading to slower browsing, higher latency (especially in remote setups), and battery drain on laptops. Running a full-fledged Virtual Machine for a single application is resource intensive. It dramatically increases boot time, CPU usage, and RAM consumption compared to a standard browser.
2. **Compatibility Issues:** - Non-seamless experiences, copying text, uploading files, or handling downloads requires manual steps to transfer safely between isolated and host environments.
3. **Implementation Hurdles:** - Early adopters faced integration issues with existing OS/browsers. For instance, Sandboxes had bugs with certain apps, and remote solutions struggled with video rendering.
4. **Trust & privacy concerns (for cloud RBI):** - Routing browsing through a third party raises data-privacy concerns: you must trust the isolation provider with page content, credentials, etc. This is why many organizations prefer local isolation.
5. **Complexity: safe file transfer:** - Downloads are tricky: you must decide how/when to allow a downloaded file onto the host, how to sanitize it, and how to present it to users without breaking workflows.

• Project description

▪ Project Overview

I am planning to develop a specialized web browser application called the **Isolated Browser with File Monitoring**. My project will incorporate advanced isolation techniques to enhance security during web browsing activities.

Running the browser within a lightweight virtualized environment similar to a simple virtual machine (VM) or container to prevent attackers from gaining unauthorized access to the host system.

Additionally, the system will include mechanisms to monitor and analyze downloaded files in real-time, restricting their interactions with the main filesystem of the user's computer. This approach addresses growing concerns in cybersecurity, where web browsers serve as primary vectors for malware, phishing, and other exploits.

using open-source tools and technologies. It will be developed primarily for desktop operating system (Windows).

The goal is to create a proof-of-concept prototype that demonstrates improved security without significantly compromising usability or performance. Isolating the browser and monitoring post-download file behaviors, my project aims to provide a safer browsing experience, particularly for users handling sensitive data or operating in high-risk tasks.

▪ ARCHITECTURE

Project Architecture has few components,

- A core isolation layer
- browser engine wrapper
- file monitoring subsystem
- interface for configuration and reporting.

I plan to leverage existing browser engines (Chromium) to avoid building web browsers from scratch and focus on the security features. The isolation will use lightweight virtualization tools (Docker containers or Linux namespaces) for efficiency, rather than full VMs like VirtualBox. For file monitoring, integration with system call tracing tools or behavioral analysis libraries will enable detection of suspicious activities, such as unauthorized file modifications or network connections initiated by downloads.

■ **Project Objectives**

My objectives of this project are to solve specific problems in browser security. Below, I outline the primary objectives.

1. *Implement Browser Isolation (Lightweight Virtualization)*

The primary goal is to encapsulate the entire browser runtime within an isolated environment that mimics simple VM. This isolation will prevent web-based threats such as malicious scripts, exploits in rendering engines, or compromised extensions from escaping the browser and affecting the host operating system.

For instance, if a user visits a site with drive-by malware, the attack would be confined to the isolated space, unable to access sensitive files, processes, or hardware on the main computer.

2. *Develop File Download Monitoring and Access Restriction Mechanisms*

The system will intercept and analyze its behavior in a quarantined area before allowing integration with the main filesystem. This includes monitoring actions like file execution, modifications to other files, or outbound network connections.

For example, if a downloaded executable attempts to write to system directories, it will be flagged and blocked, with options for user intervention or automatic deletion.

3. *Ensure Performance Optimization and Compatibility*

The isolation and monitoring features must not degrade browsing speed excessively. This involves optimizing resource usage, supporting common web standards, and ensuring compatibility with popular sites and extensions. Overhead from virtualization can deter adoption, for example, full VMs add 20-50% latency. By opting for lightweight alternatives, the project aims for near-native performance.

■ **Project Keywords**

1. Virtualization
2. Containerization
3. Sandboxing
4. Malware Detection
5. File System Isolation
6. Behavioral Analysis
7. Cybersecurity
8. Web Threats
9. Download Monitoring
10. Lightweight VM
11. System Call Tracing
12. User Privacy
13. Threat Containment

• Research Gap

In the domain, isolation techniques and malware detection in file downloads, extensive research has been conducted over the past decade.

The existing solutions have few limitations, and by examining key works on browser isolation, sandboxing, and download threat mitigation, this section highlights gaps in **performance, usability, integration of features, and accessibility**. This project differentiates itself by **offering a lightweight, local Isolation that combines browser isolation with proactive file behavior analysis**, targeting enterprise environments. (Gogri, 2023)

▪ Identified Gaps in Existing Solutions

Performance and Usability Overhead: - RBI solutions often degrade user experience due to latency, high bandwidth demands, and sluggish rendering. Pixel-pushing techniques, common in early RBI, Local sandboxes, while lighter, still impose overhead without full VM-like isolation, as critiqued in Chrome's process study. Few works address optimizing for end-users, leading to low adoption outside enterprises. (sans, n.d.)

Trust & privacy concerns (for cloud RBI): - Routing browsing through a third party raises data privacy concerns. You must trust the isolation provider with page content, credentials, etc.

Implementation Hurdles: - integration issues with existing OSes/browsers. For instance, Sandboxes had bugs with certain apps, and remote solutions struggled with video rendering.

Complex safe file transfer: - Downloads are tricky, you must decide how/when to allow a downloaded file onto the host, how to sanitize it, and how to present it to users without breaking workflows. (Anon., 2021)

Compatibility Issues: - complain about non-seamless experiences (copying text, uploading files, or handling downloads) requires manual steps to transfer safely between isolated and host environments. The delay from a remote server, even if milliseconds, can frustrate users accustomed to instant feedback.

● **Requirements Analysis**

For this project I should have good Understanding about areas like containerization, system-level programming, and system security. However, I do not necessitate any special hardware devices. Below, I break this down knowledge (skills and prerequisites) for this project.

▪ **Skills and Prerequisites**

Containerization and Virtualization Fundamentals: - Understanding Docker Firecracker Gvisor and user space sandbox To Virtualize the browser.

Browser Security Concepts: - Browser security concepts have many skilled concepts like fingerprinting hardware analyzations and GPU rendering concept cookies and authentication tokens Those techniques uh will depend on the choosing browser framework I mostly Firefox and the Chromium to this project because it has a huge community and documentation.

System level Programming: - To get Access and monitor the files and download in and use the interactions I should have better understanding of low-level program because it's give control over the virtual machines or a container to handle the situations.

System Monitoring and Analysis Tools: - So so my idea is to not build file mountain system or analysis tool from this scratch They have a few open source tools so my idea is to use that open source tool to get what I need and achieve my targets so they have a lot of open source tools so I will use that tools based on my needs.

- **Finance**

I decided to use open-source technology for this project because it's given me an opportunity to work with other open-source developers and understand how the open-source tool works and for this project I don't have any financial Documentary.

• Time Frame / Timeline

- **Week 1:** - Requirements, threat model finalization, survey & detailed design (diagram + interfaces).
- **Week 2–3:** - Environment setup: host, Firecracker/KVM toolchain, minimal headless Chromium. Create a reproducible dev VM image.
- **Week 4–5:** - Implement Host Controller (start/stop micro-VMs, lifecycle) and basic UI wrapper (open a session, show remote screen/preview).
- **Week 6–7:** - Implement secure I/O channels (virtio-vsock or v-sock proxy) for control and file transfer. Implement download interception into VM.
- **Week 8–9:** - Implement sanitizer pipeline inside micro-VM: file type detection, Clam AV scan, document conversion (e.g., DOCX→PDF), metadata extraction. Policy engine to approve/deny/transform.
- **Week 10:** - Integrate host-side transfer: sanitized artifact transfer, user prompts, logging and audit.
- **Week 11:** - Testing & evaluation: performance benchmarks, compatibility tests, malware-sample lab tests (in isolation).
- **Week 12:** - Write report, video demo, prepare final presentation.



• **Referencing / Bibliography**

Bibliography

Anon., 2021. *researchgate*. [Online]

Available at:

<https://www.researchgate.net/publication/357525867> SOK On the Analysis of Web Browser Security

Gogri, D., 2023. *Remote Browser Isolation: A Path to Zero Trust Security in the Modern Enterprise*. [Online]

Available at:

<https://www.researchgate.net/publication/385078114> Remote Browser Isolation A Path to Zero Trust Security in the Modern Enterprise

sans, n.d. *sans*. [Online]

Available at: <https://www.sans.org/white-papers/40395>